TECHNICAL DOCUMENT

The objective of this document is to provide a comprehensive technical explanation of the RAG-based Customer Support Assistant. It describes the system architecture, design decisions, and implementation details in a structured manner, enabling to understand, maintain, and extend the system effectively.

1. INTRODUCTION

1.1 What is RAG (Retrieval-Augmented Generation)

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with natural language generation. Instead of relying solely on a language model's internal knowledge, RAG systems retrieve relevant information from an external data source and use it as context to generate responses.

In this approach, user queries are first converted into embeddings and matched against a vector database to retrieve the most relevant documents. These retrieved documents are then provided as input to a Large Language Model (LLM), which generates a response grounded in the retrieved context. This ensures that the output is both accurate and domain-specific.

1.2 Why RAG is Needed

Traditional language models generate responses based only on their pre-trained knowledge, which can lead to outdated or incorrect information. They may also produce hallucinated responses when they lack sufficient knowledge about a query. RAG addresses these limitations by incorporating real-time retrieval from a custom knowledge base. This provides several advantages:

- **Improved Accuracy:** Responses are based on actual data rather than assumptions
- **Domain Adaptability:** The system can be tailored to specific domains using custom documents
- **Reduced Hallucination:** Grounding responses in retrieved context minimizes incorrect outputs

- **Up-to-Date Information:** The knowledge base can be updated without retraining the model

These benefits make RAG particularly suitable for applications where correctness and reliability are critical.

1.3 Use Case Overview

The system is designed as a **Customer Support Assistant** that helps users by answering queries based on a predefined knowledge base stored in PDF format. It is capable of handling common support-related questions such as product information, policies, and general inquiries.

When a user submits a query, the system analyzes the intent and determines whether it can generate an automated response using the RAG pipeline. If the query is straightforward and supported by the knowledge base, the system retrieves relevant information and generates a context-aware answer. However, if the query is complex, ambiguous, or requires human judgment (e.g., complaints or refund requests), it is escalated to a human agent through a Human-in-the-Loop (HITL) mechanism.

This hybrid approach ensures efficient handling of routine queries while maintaining reliability and accuracy for critical or complex cases. It makes the system suitable for real-world customer support environments where both automation and human intervention are necessary.

2. SYSTEM ARCHITECTURE EXPLANATION

2.1 Detailed Explanation of High-Level Design (HLD)

The system architecture follows a **Retrieval-Augmented Generation (RAG)** design pattern combined with a **graph-based workflow** to enable intelligent and controlled query processing. At a high level, the system is divided into two main phases: **offline knowledge processing** and **online query handling**.

In the offline phase, the PDF knowledge base is processed to prepare it for efficient retrieval. The document is first loaded and its text is extracted, after which it is divided into smaller chunks to ensure better granularity. These chunks are then converted into embeddings using an embedding model. The resulting vectors are stored in a vector database, specifically ChromaDB, which enables fast similarity-based search. This phase is performed once during initialization and creates the knowledge foundation for the system.

In the online phase, the system processes user queries in real time. When a query is received, it is passed into a graph-based workflow implemented using LangGraph. The workflow begins with intent detection, which determines whether the query can be handled automatically or requires escalation. If the query is suitable for automated processing, it follows the RAG pipeline where relevant document chunks are retrieved from the vector database. These retrieved chunks are then passed to the Large Language Model (LLM), which generates a context-aware response.

If the query is classified as complex or sensitive, it is routed to the Human-in-the-Loop (HITL) module instead of the RAG pipeline. This ensures that the system does not produce unreliable or inappropriate responses. The final output is then returned to the user, either as an automated answer or as an escalation message. This high-level design ensures separation of concerns, efficient processing, and flexibility in handling different types of queries.

2.2 Component Interactions

The system is composed of multiple components that interact in a well-defined sequence to process data and generate responses.

The interaction begins with the **user interface or API layer**, where the user submits a query. This query is passed to the **Graph Execution Module**, which orchestrates the workflow. The first component to act on the query is the **Intent Detection Node**, which analyzes the query and updates the system state with the detected intent.

Based on the intent, the workflow branches into two possible paths. In the RAG path, the query is forwarded to the **Retrieval Module**, which converts the query into an embedding and performs a similarity search in ChromaDB to fetch relevant document chunks. These

chunks are then passed to the **Query Processing Module**, where the LLM generates the final response using the retrieved context.

In the escalation path, the query is sent directly to the **HITL Module**, which handles the request by involving a human agent. This may include generating an escalation message, logging the query, or forwarding it to a support system. Throughout this process, a shared state object is maintained and updated by each component. This ensures consistent data flow and allows each module to access the necessary information. The final response, whether generated automatically or through human intervention, is returned to the user via the interface layer.

3. DESIGN DECISIONS

The design of the RAG-based Customer Support Assistant involves several critical decisions that directly impact system performance, accuracy, and scalability. These decisions include how the document is chunked, how embeddings are generated, how retrieval is performed, and how prompts are structured for the language model. Each choice is made to balance efficiency, relevance, and response quality.

3.1 Chunk Size Choice

The system uses a fixed-size chunking strategy to divide the document into smaller segments. The chunk size is chosen to ensure that each segment contains sufficient contextual information while remaining small enough for efficient retrieval. Typically, a moderate chunk size (e.g., 500–1000 tokens) is used, often with overlap between consecutive chunks.

The reason for this choice is to maintain a balance between context and precision. Larger chunks may include unnecessary information and reduce retrieval accuracy, while very small chunks may lose important context. Overlapping ensures that important information spanning across chunk boundaries is preserved. This improves the quality of retrieved results and ultimately enhances the generated responses.

3.2 Embedding Strategy

The system uses a semantic embedding approach, where each text chunk is converted into a high-dimensional vector representation. These embeddings capture the meaning of the text rather than just keyword presence, enabling more accurate similarity matching.

The embeddings are stored in a vector database such as ChromaDB, allowing efficient indexing and retrieval. This strategy is chosen because it enables the system to handle variations in user queries. Even if the query uses different wording than the document, the embedding model can still identify semantically similar content. This significantly improves the robustness and flexibility of the retrieval process.

3.3 Retrieval Approach

The system follows a similarity-based retrieval approach, where the user query is converted into an embedding and compared against stored vectors in the database. The top-k most similar chunks are selected and passed to the LLM as context.

This approach is chosen because it ensures that only the most relevant information is used for response generation. By limiting the number of retrieved chunks, the system avoids overwhelming the LLM with unnecessary data, which helps reduce latency and improve response clarity. Additionally, this method is scalable and performs efficiently even as the size of the knowledge base increases.

3.4 Prompt Design Logic

The prompt design plays a crucial role in guiding the behavior of the LLM. In this system, the prompt is structured to explicitly instruct the model to generate answers based only on the retrieved context. It typically includes:

- A role definition (e.g., “You are a customer support assistant”)
- The retrieved context from the knowledge base
- The user’s query

This design ensures that the model remains focused and generates accurate, context-aware responses. By restricting the model to the provided context, the system reduces hallucination and improves reliability. The prompt is also kept concise to minimize token usage and reduce response latency.

4. WORKFLOW EXPLANATION

4.1 LangGraph Usage

The workflow of the system is implemented using LangGraph, which enables the application to be modeled as a state-driven graph. Instead of following a fixed linear pipeline, LangGraph allows defining nodes and conditional transitions, making the workflow dynamic and adaptable.

In this system, LangGraph is used to:

- Define processing steps as nodes
- Control execution flow using edges
- Enable conditional routing based on query intent
- Maintain a shared state across all steps

The graph starts at the intent detection node and dynamically routes the query to either the RAG pipeline or the Human-in-the-Loop (HITL) path. This approach ensures efficient execution by avoiding unnecessary processing and supports modular system design.

4.2 Node Responsibilities

Each node in the workflow is responsible for a specific task, ensuring separation of concerns and modularity.

- **Intent Detection Node**
  This node is the entry point of the workflow. It analyzes the user query and classifies it into either “rag” (automated response) or “escalate” (human intervention). It updates the state with the detected intent and determines the next step in the workflow.
- **RAG Node**
  This node handles the core functionality of the system. It retrieves relevant document chunks from the vector database and generates a response using the LLM. It updates the state with the retrieved context and the final answer.
- **Human Node (HITL)**
  This node handles escalation cases. Instead of performing retrieval or generation, it

returns a predefined message indicating that the query requires human assistance. In a production system, this node can be extended to integrate with support systems or ticketing platforms.

Each node operates independently but contributes to the overall workflow by updating the shared state.

4.3 State Transitions

The workflow is driven by a shared state object that carries data across nodes. The state typically includes fields such as the user query, detected intent, retrieved context, and generated answer.

The state transitions occur as follows:

1. **Initial State**
   The state is initialized with the user query.
2. **After Intent Detection**
   The intent detection node updates the state with the intent field, indicating whether the query should follow the RAG path or be escalated.
3. **RAG Path Transition**
   If the intent is “rag”, the state is passed to the RAG node, which updates it with:
   - context (retrieved document chunks)
   - answer (generated response)
4. **HITL Path Transition**
   If the intent is “escalate”, the state is passed to the human node, which updates it with a predefined escalation response.
5. **Final State**
   The workflow terminates with the state containing the final answer, which is returned to the user.

This state-driven approach ensures consistency and traceability throughout the workflow, allowing each node to access and update relevant information as needed.

5. CONDITIONAL LOGIC

Conditional logic is a core part of the system that enables intelligent decision-making during query processing. It determines how each user query should be handled by analyzing its intent and routing it through the appropriate path in the workflow. This ensures that simple queries are answered automatically while complex or sensitive queries are escalated to a human agent.

5.1 Intent Detection

Intent detection is the first step in the conditional logic process. It analyzes the user query to classify it into predefined categories that guide further processing. In this system, intent detection is implemented using a rule-based approach. The query is converted to lowercase and checked for specific keywords such as *refund*, *complaint*, *agent*, or *human*. If any of these keywords are present, the query is classified as **"escalate"**; otherwise, it is classified as **"rag"**.

This approach is simple, efficient, and suitable for initial implementations. It ensures quick decision-making without requiring additional computational overhead. However, it can be extended in the future using machine learning or LLM-based classification for more accurate intent detection.

5.2 Routing Decisions

Routing decisions are made based on the detected intent and determine the path that the query will follow in the workflow. This is implemented using conditional edges in the graph, which direct the flow dynamically.

- If the intent is **"rag"**, the query is routed to the RAG node. In this path, the system retrieves relevant information from the vector database and generates a response using the LLM.
- If the intent is **"escalate"**, the query is routed to the Human-in-the-Loop (HITL) node. In this case, the system avoids automated response generation and instead prepares the query for human handling.

This routing mechanism ensures efficient use of resources by avoiding unnecessary LLM calls for queries that require human intervention. It also improves reliability by ensuring that complex or sensitive queries are handled appropriately.

6. HITL IMPLEMENTATION

The Human-in-the-Loop (HITL) implementation introduces human intervention into the system to handle cases where automated responses may not be reliable or appropriate. It acts as a fallback mechanism within the workflow, ensuring that the system maintains accuracy, trust, and robustness in real-world customer support scenarios.

6.1 Role of Human Intervention

Human intervention is triggered when the system detects that a query cannot be confidently handled using the RAG pipeline. In the current implementation, this decision is made during the intent detection stage, where certain keywords (such as *refund*, *complaint*, or *agent*) indicate that the query requires escalation. When such a query is identified, it is routed to the HITL node instead of the RAG node. The system then generates a response indicating that the request will be handled by a human agent. In a production environment, this step can be extended to integrate with external systems such as ticketing platforms, customer support dashboards, or notification systems.

The role of human intervention is therefore to:

- Handle complex, ambiguous, or sensitive queries
- Provide accurate and context-aware responses when automation is insufficient
- Ensure compliance with business rules and customer service policies

6.2 Benefits

The HITL approach provides several important advantages:

- **Improved Accuracy:**
  Human agents can handle edge cases and complex queries more effectively than automated systems.

- **Reduced Risk of Errors:**
  By avoiding automated responses in uncertain situations, the system minimizes incorrect or misleading outputs.
- **Enhanced User Trust:**
  Users are more likely to trust the system when they know that critical issues are handled by humans.
- **Better Handling of Sensitive Cases:**
  Queries involving complaints, refunds, or emotional context are better managed by human agents.
- **Continuous Improvement:**
  Human responses can be logged and used to improve the knowledge base and system performance over time.

6.3 Limitations

Despite its advantages, HITL also introduces certain challenges:

- **Increased Response Time:**
  Human intervention is slower compared to automated responses, which may affect user experience.
- **Scalability Constraints:**
  Handling a large number of escalated queries requires sufficient human resources.
- **Operational Cost:**
  Maintaining a human support team increases operational expenses.
- **Dependency on Human Availability:**
  System performance for escalated queries depends on the availability and efficiency of human agents.

7. CHALLENGES & TRADE-OFFS

7.1 Retrieval Accuracy vs Speed

One of the primary trade-offs in the system is between retrieval accuracy and response speed. Higher retrieval accuracy can be achieved by searching a larger number of document chunks or using more complex similarity calculations. However, this increases computation time and latency. To optimize performance, the system typically limits retrieval to the top-k most relevant chunks. While this improves speed, it may sometimes exclude useful context, slightly reducing accuracy. Conversely, increasing the number of retrieved chunks improves the chances of finding relevant information but can slow down the system and increase the load on the LLM.

Therefore, an optimal balance is required where enough relevant data is retrieved without significantly impacting response time.

7.2 Chunk Size vs Context Quality

The choice of chunk size directly impacts both retrieval performance and context quality. Larger chunks contain more information, which helps preserve context and improves the quality of responses. However, they may also include irrelevant content, reducing retrieval precision. Smaller chunks, on the other hand, improve retrieval accuracy by focusing on specific pieces of information. However, they may lose context, leading to incomplete or fragmented responses.

To address this trade-off, the system uses moderately sized chunks with overlap. This approach balances context preservation and retrieval precision, ensuring that important information is retained while maintaining efficient search performance.

7.3 Cost vs Performance

Another important trade-off is between system cost and performance. High-performance models and extensive retrieval operations can significantly improve response quality but also increase operational costs.

For example:

- Using larger or more advanced LLMs improves response quality but increases API costs and latency
- Retrieving more chunks or using complex embeddings increases computation cost
- Frequent API calls can lead to higher expenses in production environments

To manage this, the system uses optimized strategies such as:

- Limiting the number of retrieved chunks
- Designing concise prompts to reduce token usage
- Using efficient models where possible

These optimizations help maintain a balance between delivering high-quality responses and controlling operational costs.

8. TESTING STRATEGY

The testing strategy ensures that the RAG-based Customer Support Assistant functions correctly, produces accurate responses, and handles different types of queries reliably. It focuses on validating each component of the system as well as the end-to-end workflow. The testing approach includes both functional and scenario-based testing to evaluate performance under real-world conditions.

8.1 Testing Approach

The system is tested at multiple levels to ensure correctness and robustness:

Unit Testing:

Each module is tested independently to verify its functionality. For example:

- Intent detection is tested with different types of queries to ensure correct classification
- Retrieval module is tested to confirm that relevant document chunks are returned
- RAG node is tested to verify proper response generation

This helps identify issues at the component level before integration.

Integration Testing

Integration testing ensures that different modules work together correctly. The focus is on validating:

- Data flow between modules
- Proper state updates across nodes
- Correct execution of the graph workflow

This step confirms that the overall pipeline functions as expected when components interact.

End-to-End Testing

End-to-end testing evaluates the complete system from user input to final output. It ensures that:

- Queries are correctly routed based on intent
- Responses are generated accurately using retrieved context
- Escalation is triggered when required

This testing simulates real user interactions and validates the system's behavior in practical scenarios.

Edge Case Testing

Special cases are tested to ensure system robustness, such as:

- Empty or invalid queries
- Queries with no relevant information in the knowledge base
- Complex or ambiguous queries
- LLM failure or timeout scenarios

Handling these cases properly ensures reliability and prevents system failures.

8.2 Sample Queries

The system is tested using a variety of queries to cover different scenarios:

RAG-Based Queries (Automated Response)

- “What is the pdf about?”
- “What is a deadlock ?”
- “What is the summary of the knowledge pdf?”

These queries are expected to retrieve relevant context and generate accurate answers.

Escalation Queries (HITL)

- “Can I talk to a human agent?”
- “What is a refund policy?”

These queries should be routed to the HITL module instead of generating automated responses.

Edge Case Queries

- “” (empty query)
- “asdfghjkl” (irrelevant input)

These help test system stability and error handling mechanisms.

9. FUTURE ENHANCEMENTS

9.1 Multi-Document Support

The current system is designed to process a single PDF knowledge base. In future versions, it can be extended to support multiple documents from different sources such as PDFs, web pages, and databases. This would allow the system to handle a broader range of queries and provide more comprehensive responses. Implementing document tagging and filtering mechanisms can further improve retrieval accuracy across multiple sources.

9.2 Feedback Loop

A feedback mechanism can be introduced to continuously improve system performance. Users can rate responses or provide corrections, which can be logged and analyzed. This feedback can be used to refine retrieval strategies, improve prompt design, and update the knowledge base. Over time, this creates a self-improving system that adapts to user needs and reduces errors.

9.3 Memory Integration

Currently, the system processes each query independently without retaining conversation history. Adding memory capabilities would allow the system to maintain context across multiple interactions. This would enable more natural, conversational experiences where the system can understand follow-up questions and provide more personalized responses. Memory can be implemented using session-based storage or conversational memory frameworks.

9.4 Real Human Agent Interaction (Enhanced HITL)

In the current implementation, escalation returns a predefined message. In future enhancements, this can be upgraded to enable real-time interaction with a human agent. When a query is escalated, the system can connect the user to a live support representative through chat or messaging platforms. This may involve integration with customer support tools such as ticketing systems or live chat services. This enhancement significantly improves user experience by providing immediate and personalized assistance.

9.5 Deployment

For real-world usage, the system can be deployed on cloud platforms such as AWS, Azure, or GCP. Deployment involves containerizing the application using Docker, setting up APIs, and configuring scalable infrastructure. Features like load balancing, monitoring, and logging can be added to ensure reliability and performance. This step is essential to transition the project from a prototype to a production-grade system.

9.6 Application Development (End-to-End Product)

The system can be extended into a complete application by developing a user-friendly interface. This may include:

- A web application using frameworks like React for frontend and FastAPI for backend
- A mobile application for on-the-go access
- A chat-based interface similar to modern customer support systems

This transformation would make the project production-ready, allowing real users to interact with the system seamlessly. It also opens possibilities for deployment on cloud platforms, enabling scalability and accessibility.